



Projeto Sonora - Fase 01: Fundamentos de POO

1. A ideia geral (leia com atenção, isto vale para o semestre inteiro)

A partir de agora você vai construir um sistema só, que cresce a cada aula. Este é o **Sonora**, uma plataforma de streaming de músicas (pense num Spotify bem enxuto, sem a parte bonita, só o motor por baixo).

Cada nova fase do projeto chega como um enunciado novo, e sempre funciona assim:

1. Você clona a pasta/projeto da fase anterior para uma pasta/projeto novo (ex.: sonora-fase01 vira sonora-fase02).
2. A fase nova é um projeto/repositório independente (a fase anterior fica intacta, congelada).
3. Você implementa o que o novo enunciado pede, sem quebrar o que já funcionava.

Ou seja: o que você entregar mal feito hoje vai te perseguir nas próximas fases. Capriche na base.

Esta é a **Fase 01**, então não há o que clonar ainda: você começa do zero. Nome da pasta: sonora-fase01.

Nesta fase o foco é montar a estrutura do sistema com classes bem feitas. As validações de dados e o tratamento de erros virão na Fase 02, quando estudarmos exceções, e você vai voltar nestas mesmas classes para deixá-las robustas.

2. O que você PODE e o que você NÃO PODE usar nesta fase

Esta fase é proposital: ela obriga você a resolver tudo sem coleções e sem herança, porque esses assuntos ainda não foram vistos. Isso não é limitação, é treino.

Pode usar (e vai precisar):

- Classes, atributos encapsulados (`private`), construtores, `this`
- Getters e setters
- Membros de classe (`static`) para gerar identificadores
- Sobrecarga de métodos
- vetores de tamanho fixo para armazenar os dados
- Scanner para ler do teclado

NÃO pode usar nesta fase:

- ArrayList, List, Map, Set ou qualquer coleção do java.util para guardar os dados (use arrays)
- Herança (extends) ou interfaces (implements), isso vem nas próximas unidades
- Leitura ou escrita de arquivos, isso vem na unidade de Persistência
- Bibliotecas externas

Ainda não precisa usar: throw, try, catch, finally. Exceções são o assunto da Fase 02. Aqui, quando uma operação não puder ser concluída (por exemplo, adicionar numa estrutura cheia), você sinaliza isso pelo valor de retorno do método, como descrito na seção 3.

3. As classes do sistema

Você vai implementar cinco classes. As assinaturas abaixo são obrigatórias (vou corrigir chamando esses métodos). Você pode criar métodos auxiliares privados à vontade.

3.1. Classe Musica

Representa uma faixa do acervo.

Atributo	Tipo	Significado
id	int	Gerado automaticamente, sequencial e único, via contador static. O primeiro cadastro é 1.
titulo	String	Título da faixa.
artista	String	Nome do artista.
duracaoSegundos	int	Duração em segundos.
reproducoes	int	Quantas vezes foi tocada. Começa em 0. Só aumenta, nunca é definido de fora.

Métodos obrigatórios:

```
public Musica(String titulo, String artista, int duracaoSegundos)
public int getId()
public String getTitulo()
public String getArtista()
public int getDuracaoSegundos()
public int getReproducoes()
public void reproduzir()           // incrementa reproducoes em 1
```



```
public String getDuracaoFormatada() // devolve "mm:ss", ex.: 354  
segundos vira "05:54"
```

Regras de projeto:

- O id é atribuído sozinho no momento em que a música é criada, usando um contador static da classe. Não existe setId público.
- reproducoes começa em 0 e só muda através de reproduzir(). Não existe setReproducoes público.
- getDuracaoFormatada() deve devolver minutos e segundos com dois dígitos cada, com zero à esquerda quando necessário (por exemplo, 65 segundos vira "01:05").

3.2. Classe Usuario

Quem usa a plataforma.

Atributo	Tipo	Significado
id	int	Sequencial e único, com contador static próprio (independente do de Musica).
nome	String	Nome do usuário.
email	String	E-mail do usuário.

Métodos obrigatórios: construtor Usuario(String nome, String email), os getters correspondentes, e nenhum setId.

3.3. Classe Playlist

Uma lista de músicas de um usuário. Capacidade fixa de 100 músicas (array).

Métodos obrigatórios:

```
public Playlist(String nome, Usuario dono)  
public String getNome()  
public Usuario getDono()  
public int getQuantidade() // quantas músicas tem agora  
public boolean adicionar(Musica musica) // adiciona no fim; ver regra  
de retorno  
public Musica getNaPosicao(int indice) // devolve a música da  
posição; ver regra de retorno  
public boolean removerNaPosicao(int indice) // remove e reorganiza; ver  
regra de retorno  
public int getDuracaoTotalSegundos() // soma das durações das  
músicas
```



```
public void reproduzirTudo()           // chama reproduzir() em cada
música
```

Regras de projeto (sinalização por valor de retorno, sem exceções nesta fase):

- adicionar devolve true se a música foi adicionada e false se não foi possível (música nula ou playlist cheia).
- getNaPosicao devolve a música daquela posição, ou null se o índice estiver fora do intervalo válido.
- removerNaPosicao devolve true se removeu e false se o índice era inválido. Ao remover, não deixe “buraco” no array: reorganize para os itens ficarem contíguos.
- Uma mesma música pode aparecer em playlists diferentes (é a mesma referência).

3.4. Classe Plataforma

A gerenciadora central. Guarda o acervo de músicas e os usuários, cada um em seu array de capacidade fixa 500.

Métodos obrigatórios:

```
public boolean cadastrarMusica(Musica musica) // false se nula ou
acervo cheio
public boolean cadastrarUsuario(Usuario usuario) // false se nulo ou
acervo cheio
public Musica buscarMusicaPorId(int id)        // sobrecarga (int);
null se não encontrar
public Musica buscarMusica(String titulo)     // sobrecarga (String);
null se não encontrar
public int getTotalMusicas()
public int getTotalUsuarios()
```

Repare que buscarMusicaPorId(int) e buscarMusica(String) têm o mesmo nome de intenção mas listas de parâmetros diferentes. Isso é sobrecarga, e é obrigatório fazer assim (não invente buscarMusicaPorTitulo).

3.5. Classe App

A classe com o public static void main. É o cliente do sistema: um menu de console que integra tudo.

Como ainda não há leitura de arquivo, para conseguir testar rápido você pode criar algumas músicas na mão no início do main (chamando o construtor de Musica e cadastrarMusica). Isso é só para popular o acervo de teste.



Menu mínimo (você pode enriquecer):

```
=== Sonora ===  
1 - Cadastrar música manualmente  
2 - Cadastrar usuário  
3 - Criar playlist e adicionar músicas  
4 - Buscar música por id  
5 - Buscar música por título  
6 - Reproduzir uma música  
7 - Listar acervo  
0 - Sair
```

Dica de robustez: leia a opção do menu de forma segura usando `Scanner.hasNextInt()` antes de `nextInt()`, para o programa não parar se o usuário digitar um texto. O tratamento completo de entrada inválida (com `try/catch`) é justamente um dos assuntos da Fase 02.

4. Comportamentos que precisam funcionar (e onde está a cobrança)

Isto aqui é o que separa um “compilou” de um trabalho bem feito:

- **Identificadores únicos por classe.** Cadastre três músicas e os ids devem sair 1, 2, 3. Cadastre dois usuários e os ids devem sair 1, 2 (contador próprio, independente do de música). Se os ids repetirem ou “vazarem” entre as classes, o `static` está errado.
- **Encapsulamento de verdade.** Todos os atributos `private`. Nada de título público acessado direto de fora. `id` e `reproducoes` sem setter público.
- **Duração formatada.** `getDuracaoFormatada()` de 354 devolve “05:54”; de 65 devolve “01:05”; de 600 devolve “10:00”.
- **Array contíguo.** Depois de remover uma música do meio da playlist, `getQuantidade()` diminui em 1 e não sobra “buraco”: as músicas seguintes andam uma posição para trás.
- **Sinalização correta.** Adicionar numa playlist cheia devolve `false` e não estoura o programa. Buscar algo que não existe devolve `null`.
- **Sobrecarga.** As duas buscas coexistem com o mesmo nome e parâmetros diferentes.

5. Roteiro de demonstração (você tem que conseguir reproduzir tudo isto)

Prepare seu App para que, numa demonstração, você consiga mostrar cada um destes comportamentos:

1. Cadastrar três músicas e listar o acervo, mostrando os ids saindo 1, 2, 3.
2. Reproduzir uma música três vezes e mostrar `getReproducoes()` retornando 3.

3. Mostrar `getDuracaoFormatada()` para 354, 65 e 600 segundos.
4. Cadastrar um usuário, criar uma playlist para ele, adicionar músicas e mostrar `getQuantidade()` e `getDuracaoTotalSegundos()`.
5. Encher uma playlist até 100 músicas e tentar adicionar a 101^a, mostrando que adicionar devolve `false` e a quantidade continua 100.
6. Remover uma música do meio da playlist e mostrar que não ficou buraco (as posições seguintes andaram para trás).
7. Buscar uma música por um id que existe e por um id que não existe (o segundo devolve `null`).
8. Buscar uma música por título (a versão sobrecarregada).
9. Chamar `reproduzirTudo()` e mostrar que a contagem de reproduções de todas as músicas da playlist subiu.

Sugestão forte: coloque esses passos como um roteiro no seu README. Na correção eu vou seguir mais ou menos essa lista.

6. Entregáveis

1. Código-fonte na pasta sonora-fase01,
2. Um diagrama de classes simples das cinco classes (atributos e métodos principais, com a visibilidade em UML: sinal de menos para privado, sinal de mais para público). Pode ser feito em ferramenta ou desenhado à mão e fotografado, desde que legível.

7. O que vem na Fase 02 (semana que vem)

Guarde bem este projeto, porque a Fase 02 começa clonando ele. Lá você vai:

- Adicionar validação nos construtores e setters (título não pode ser vazio, duração precisa estar num intervalo válido, e assim por diante), usando `throw` para impedir que um objeto nasça em estado inválido.
- Trocar parte da sinalização por valor de retorno por lançamento de exceções, e discutir o que faz sentido continuar como retorno e o que vira exceção.
- Deixar o menu do App à prova de digitação errada com `try/catch`.

Ou seja: o que você construir bem agora vai facilitar muito a sua Fase 02.

8. Erros comuns que vão te dar dor de cabeça (evite)

- Esquecer o `static` no contador de id: os ids saem errados ou repetidos.
- Deixar atributos públicos ou criar `setId` e `setReproducoes` públicos.



- Deixar “buraco” no array ao remover uma música da playlist.
- Estourar o programa ao adicionar numa playlist cheia, em vez de devolver false.
- Criar dois métodos de busca com nomes diferentes em vez de usar sobrecarga.
- getDuracaoFormatada sem o zero à esquerda (mostrando “5:54” em vez de “05:54”).